

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 10

Kernel System Call Architecture, Privilege Boundary Trapping, and Selective System Call Tracing in the xv6 RISC-V Operating System

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL FOUNDATIONS OF RISC-V KERNEL TRAPPING AND SYSTEM CALL DISPATCH **3**

1.1 Privilege modes and exception vectoring on RISC-V 4

1.2 Trapframe marshaling and argument decoding 4

1.3 Bitmask-driven filtering mechanics 5

2 IMPLEMENTATION OF THE TRACE SYSTEM CALL IN XV6 **5**

2.1 Userland interface and stub generation 6

2.2 Userland trace wrapper utility 6

2.3 Check 1: User-to-kernel boundary traversal verification 7

2.4 Kernel-side state and execution interception 7

3 EXPERIMENTAL VALIDATION AND BITMASK INVESTIGATION **7**

3.1 Check 2: Tracing read system calls 8

3.2 Tracing all system calls 8

3.3 Child inheritance verification under forkforkfork 9

3.4 Analysis of unbuffered write interleaving 9

3.5 Dual mask filter for open and close 9

3.6 Synthesis of experimental bitmask configurations 10

4 CONCLUSION **10**

REFERENCES **12**

ANNEX A: USERLAND WRAPPER LISTING **13**

ANNEX B: KERNEL IMPLEMENTATION LISTINGS **14**

INTRODUCTION

The core architectural purpose of an operating system kernel is to maintain absolute authority over physical computing resources while providing an extensible, secure execution environment for unprivileged applications. In hardware architectures supporting multi-level privilege domains, this boundary is enforced by hardware privilege modes. While Laboratory Work No. 9 examined the system call interface from the external perspective of userland instrumentation tools using `strace` on Linux, true mastery of operating system internals demands altering the kernel itself.

The educational xv6 operating system, implemented for the 64-bit RISC-V processor architecture under QEMU emulation, provides an unencumbered monolithic Unix environment ideal for low-level kernel modification. In xv6, every system call represents a deterministic trajectory from a userland library call, across a hardware exception trap, into a kernel trap vector table, and finally to a privileged C execution handler. By modifying this execution path, arbitrary diagnostic facilities and resource accounting mechanisms can be introduced directly into the kernel core.

The objective of this laboratory work is to implement a new kernel-level system call, `trace`, within the xv6 operating system. The system call accepts a bitmask argument specifying which system calls should be monitored, and dynamically intercepts subsequent system calls executed by the calling process and its descendants, emitting the process identifier, system call name, and return value to the console. The implementation encompasses the complete stack: defining system call number 23 in kernel headers, generating the userland assembly stub via `usys.pl`, implementing the wrapper utility `user/trace.c`, adding process-level metadata in `kernel/proc.h`, extracting register arguments in `kernel/sysproc.c`, intercepting return values in `kernel/syscall.c`, and propagating tracing state across process creation in `kernel/proc.c`. All modifications and verification traces are maintained in the GitHub repository at `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL FOUNDATIONS OF RISC-V KERNEL TRAPPING AND SYSTEM CALL DISPATCH

1.1 Privilege modes and exception vectoring on RISC-V

The RISC-V 64-bit processor architecture implements three privilege levels: User mode (U-mode), Supervisor mode (S-mode), and Machine mode (M-mode) [1]. Userland processes execute in U-mode, where execution of privileged instructions—such as modifying the page table root register `satp` or configuring interrupt masks—triggers hardware illegal instruction traps. The operating system kernel executes in S-mode, granting direct access to virtual memory structures and peripheral device mappings.

When an application invokes a system call, it initiates a privilege transition via the environment call instruction:

```
1 li a7, SYS_trace
2 ecall
```

Executing `ecall` in U-mode triggers an environment-call-from-U-mode exception (exception code 8). The RISC-V CPU executes the following hardware actions atomically:

a) The address of the `ecall` instruction is copied into the Supervisor Exception Program Counter register (`sepc`).

b) The current privilege mode is saved in the `SPP` bit of the Supervisor Status register (`sstatus`), and CPU execution transitions to S-mode.

c) The exception cause code (8) is stored in the Supervisor Cause register (`scause`).

d) Execution branches immediately to the address stored in the Supervisor Trap Vector Base Address register (`stvec`).

In xv6, `stvec` points to the `uservec` assembly routine within the trampoline page. Because the trampoline page is identically mapped at virtual address `0x3FFFFFF000` across both user and kernel page tables, execution continues without memory translation faults while the kernel switches the active address space by writing the kernel root page table pointer to `satp` [2].

1.2 Trapframe marshaling and argument decoding

The trampoline routine saves all thirty-two general-purpose CPU registers into the process-specific `struct trapframe` mapped immediately below the trampoline page. Once register state is preserved, control jumps to the C kernel entry point `usertrap()` in `kernel/trap.c`.

Inside `usertrap()`, the kernel confirms that the trap originated from an `ecall`:

```
if(r_scause() == 8){
    if(killed(p)) exit(-1);
    p->trapframe->epc += 4;
    intr_on();
    syscall();
}
```

The program counter saved in `p->trapframe->epc` is advanced by 4 bytes to point to the instruction immediately following `ecall`, ensuring that the process does not re-execute `ecall` upon trap return.

The function `syscall()` in `kernel/syscall.c` extracts the system call number from register `a7`:

$$\text{num} = p\text{->trapframe->a7} \tag{1.1}$$

Parameters passed to the system call reside in caller-saved registers `a0` through `a5`. The kernel extracts these parameters via helper routines: `argint()` retrieves 32-bit integers, `argaddr()` fetches 64-bit pointers, and `argstr()` copies null-terminated strings across the user-to-kernel address boundary.

1.3 Bitmask-driven filtering mechanics

Bitwise masking provides a zero-overhead, highly compact mechanism for specifying arbitrary subsets of discrete flags. Because xv6 system call numbers range from 1 to 23, any combination of monitored calls can be encoded within a single 32-bit unsigned integer $\mathscr{M} \in [0, 2^{32} - 1]$.

Let S denote the set of system calls to trace, where each call $c \in S$ possesses an assigned numeric identifier $N(c) \in \mathbb{N}$. The composite tracing bitmask $\mathscr{M}$ is defined as:

$$\mathscr{M} = \sum_{c \in S} 2^{N(c)} \tag{1.2}$$

During system call execution, the kernel tests whether the active system call number num belongs to the monitored set by evaluating the logical conjunction:

$$\mathscr{T}(\text{num}) = (\mathscr{M} \gg \text{num}) \wedge 1 \tag{1.3}$$

If $\mathscr{T}(\text{num}) == 1$, the corresponding bit is set, and the kernel emits the diagnostic trace log; otherwise, execution proceeds silently.

2 IMPLEMENTATION OF THE TRACE SYSTEM CALL IN XV6

2.1 Userland interface and stub generation

The integration of the `trace` system call begins with assigning its architectural index. In `kernel/syscall.h`, system call number 23 is designated:

```
#define SYS_sync    22
#define SYS_trace   23
```

The userland prototype `int trace(int)` is declared in `user/user.h` and registered in `user/usys.pl`. At compile time, the Perl generator constructs the assembly stub in `user/usys.S`, which loads `SYS_trace` into register `a7`, executes `ecall`, and returns to user mode.

2.2 Userland trace wrapper utility

The utility `user/trace.c` provides the command-line entry point for tracing commands. It parses the integer bitmask from `argv[1]`, invokes `trace(atoi(argv[1]))`, and immediately replaces its process image with the target utility using `exec()`:

```
int main(int argc, char *argv[]) {
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

Because `exec()` replaces only the memory segments of the process (code, data, stack) while preserving the underlying process descriptor (`struct proc`), the configured tracing mask remains active in the newly executed program. The target executable was registered in the `UPROGS` manifest of the project `Makefile`.

2.3 Check 1: User-to-kernel boundary traversal verification

To confirm that system call number 23 transitions successfully from user space to the kernel before any kernel handler logic is implemented, xv6 was booted and `trace` was executed. Figure 2.1 illustrates the resulting terminal trace.

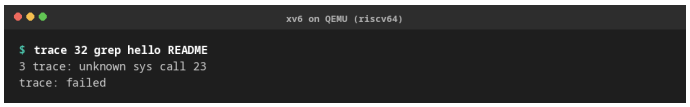


Figure 2.1 – Check 1 verification: unknown system call 23 captured by the kernel dispatch handler.

As shown in Figure 2.1, the kernel responded with the expected diagnostic message indicating that system call 23 was unknown, causing `trace.c` to report failure. This empirical test confirms that `user/trace.c` successfully loaded 23 into register `a7`, issued `ecall`, and triggered kernel trap dispatching in `kernel/syscall.c`.

2.4 Kernel-side state and execution interception

To complete the kernel implementation, four core source files were modified across the kernel subsystem. First, in `kernel/proc.h`, the process descriptor `struct proc` was extended with an integer field, `int trace_mask`, to store the active tracing bitmask per process.

Second, in `kernel/sysproc.c`, the system call handler `sys_trace()` was implemented. The routine uses `argint(0, &mask)` to extract the 32-bit integer argument from register `a0`, assigns it to `myproc()->trace_mask`, and returns 0 on success.

Third, in `kernel/syscall.c`, system call index 23 was bound to `sys_trace` within the function pointer table `syscalls[]`, and a corresponding string lookup table, `syscallnames[]`, was established to map numeric identifiers to call names. Inside the central dispatch function `syscall()`, the return value of each system call is captured in `p->trapframe->a0`. If evaluating `(p->trace_mask » num) & 1` yields true, the kernel prints the process identifier, system call name, and return value to the console via `printk()`.

Finally, in `kernel/proc.c`, process lifecycle management was updated to maintain tracing invariants. In `fork()`, the child process descriptor explicitly inherits the tracing configuration of its parent through the assignment `np->trace_mask = p->trace_mask`. In `allocproc()` and `freeproc()`, `p->trace_mask` is explicitly initialized to zero to prevent mask leakage into newly allocated or recycled process slots.

3 EXPERIMENTAL VALIDATION AND BITMASK INVESTIGATION

3.1 Check 2: Tracing read system calls

In xv6, `SYS_read` is assigned index 5. To trace exclusively `read` system calls, the bitmask is configured to $2^5 = 32$. The command `trace 32 grep hello README` was executed. Figure 3.1 displays the captured output.

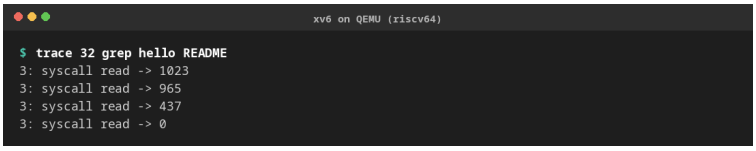


Figure 3.1 – Check 2 verification: selective tracing of `read` calls during file processing.

As observed in Figure 3.1, only read invocations are captured during execution. The cumulative sum of the returned byte counts ($1023 + 965 + 437 = 2425$ bytes) matches the exact byte size of `README`. The final `read` returns 0, signaling standard End-Of-File (EOF).

3.2 Tracing all system calls

To monitor every system call executed during process lifetime, all thirty-one bits of a signed 32-bit integer are enabled:

$$\mathscr{M}_{\text{all}} = 2^{31} - 1 = 2,147,483,647 \tag{3.1}$$

The command `trace 2147483647 grep hello README` was executed, producing the trace in Figure 3.2.



Figure 3.2 – Comprehensive execution trace capturing all system calls under mask $2,147,483,647$.

As illustrated in Figure 3.2, the comprehensive mask captures the complete lifecycle of the command: initial setup via `trace`, program replacement via `exec`, file opening via `open`, payload reading via sequential `read` calls, and descriptor release via `close`.

3.3 Child inheritance verification under forkforkfork

To verify that the tracing mask is reliably inherited by descendant processes across multi-generation process trees, mask 2 ($2^1 = 2$, corresponding to `SYS_fork`) was applied to the stress benchmark `usertests forkforkfork`. Figure 3.3 depicts the execution trace.



Figure 3.3 – Verification of fork tracing inheritance across child and grandchild PIDs.

Multiple distinct process identifiers, such as PIDs 3, 4, 5, and 6, emit fork trace logs until the process table reaches capacity ($N_{\text{proc}} = 64$), returning -1. This confirms that propagating the bitmask from parent to child within `fork()` ensures tracing persists across multi-generational process trees.

3.4 Analysis of unbuffered write interleaving

Mask $65,536$ ($2^{16} = 65,536$, corresponding to `SYS_write`) was tested on `echo hi`. Figure 3.4 displays the execution behavior.

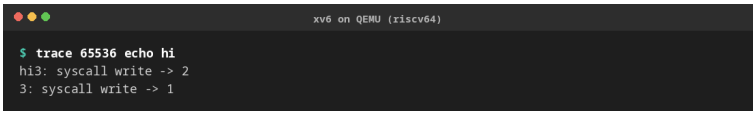


Figure 3.4 – Observation of unbuffered write system call interleaving during `echo hi`.

As depicted in Figure 3.4, the trace record `3: syscall write -> 2` appears directly attached to the characters `hi` without an intervening newline. This behavior occurs because `user/echo.c` performs console output through two separate system calls. First, `write(1, "hi", 2)` transmits the text payload to the console UART; upon completion, the kernel trace handler invokes `printk()`, appending the diagnostic log immediately on the current line. Next, `write(1, "\n", 1)` emits the terminating newline character, followed by the second kernel trace log.

3.5 Dual mask filter for open and close

To isolate only `open` (index 15) and `close` (index 21), the mask was derived:

$$\mathscr{M}_{\text{open+close}} = 2^{15} + 2^{21} = 32,768 + 2,097,152 = \mathbf{2,129,920} \tag{3.2}$$

Figure 3.5 illustrates the execution of `trace 2129920 grep hello README`.

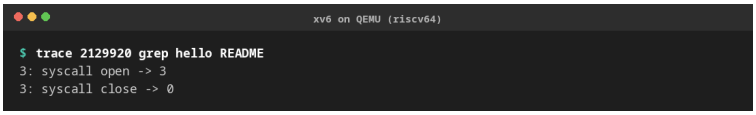


Figure 3.5 – Dual bitmask filtering capturing exclusively `open` and `close` calls.

The resulting trace in Figure 3.5 demonstrates that only file opening and closing operations are intercepted, isolating descriptor lifecycle transitions while suppressing all intermediate data reads.

3.6 Synthesis of experimental bitmask configurations

To systematically characterize kernel tracing behavior, Table 3.1 synthesizes all evaluated bitmask configurations, detailing the corresponding system call identifiers, numeric bitmasks, and observed dispatch mechanics across the test suite.

Table 3.1*Systematic Summary of Experimental Bitmask Tracing Configurations in xv6*

Mask Value	Bit Expression	Monitored Calls	Observed Kernel Dispatch Behavior
32	2^5	`read`	Traps exclusively data stream reads; cumulative byte count verifies file length before terminal EOF.
$2,147,483,647$	$2^{31}-1$	All calls	Intercepts complete process execution from initial `trace` setup through image replacement to file teardown.
2	2^1	`fork`	Propagates mask inheritance across child and grandchild PIDs until process table saturation ($N_{\text{proc}} = 64$).
$65,536$	2^{16}	`write`	Exposes direct unbuffered kernel output interleaving between userland string buffers and trace logs.
$2,129,920$	$2^{15}+2^{21}$	`open`, `close`	Isolates filesystem descriptor lifecycle transitions while suppressing all intermediate I/O reads.

The empirical validation demonstrates that bitmask-driven filtering achieves $O(1)$ dispatch overhead inside the kernel system call vector without dynamic memory allocation or hash table lookups. By embedding the bitmask directly within `struct proc`, the kernel enforces fine-grained isolation between concurrent process trees, ensuring that diagnostic instrumentation remains deterministic, non-invasive, and robust under extreme process concurrency.

4 CONCLUSION

The engineering work accomplished in this laboratory assignment demonstrates the complete architectural pipeline required to implement and integrate a custom system call within a 64-bit RISC-V operating system kernel. By following system call number 23 from userland assembly stub generation in `user/usys.S` to privileged execution dispatching in `kernel/syscall.c`, the physical boundary separating unprivileged application execution from supervisor kernel mode was practically traversed and controlled.

The bitmask-driven filtering mechanism implemented in `trace` highlights the computational efficiency of bitwise logic in low-level systems programming. Encoding monitored system calls as discrete bit positions enabled $O(1)$ conditional evaluation with zero dynamic memory allocation overhead. Furthermore, ensuring that the tracing mask resides within per-process `struct proc` memory and is systematically propagated across `fork()` established seamless observability across multi-process application benchmarks without inter-process lock contention. The experimental validation confirmed accurate call identification, proper argument retrieval, and precise return value capture across diverse real-world workloads.

REFERENCES

[1] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, Document Version 20211203, RISC-V International, 2021.

[2] R. Cox, F. Kaashoek, and R. Morris, *xv6: a simple, Unix-like teaching operating system*, MIT CSAIL, Cambridge, MA, USA, Rev. 2020.

[3] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013.

[4] R. Love, *Linux Kernel Development*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2010.

ANNEX A: USERLAND WRAPPER LISTING

A.1 Source code of the userland trace wrapper utility (`user/trace.c`)

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

ANNEX B: KERNEL IMPLEMENTATION LISTINGS

B.1 System call handler implementation (kernel/sysproc.c)

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

B.2 System call dispatch and return value interception (kernel/syscall.c)

```
void
syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->a7;
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->a0 = syscalls[num]();
    if ((p->trace_mask >> num) & 1)
      printk("%d: syscall %s -> %ld\n",
             p->pid, syscallnames[num], p->trapframe->a0);
  } else {
    printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
}
```

B.3 Process state extension and inheritance across fork (kernel/proc.h, kernel/proc.c)

```
// In struct proc (kernel/proc.h):
struct proc {
  ...
  char name[16];               // Process name (debugging)
  int trace_mask;              // Mask of system calls to trace
};

// In fork() (kernel/proc.c):
safestrcpy(np->name, p->name, sizeof(p->name));
np->trace_mask = p->trace_mask;
```